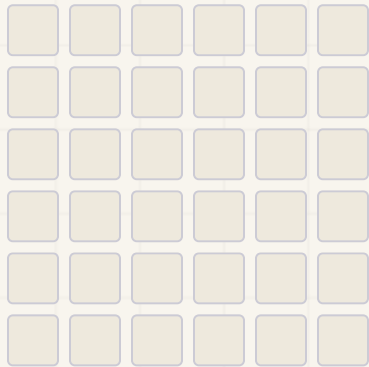


Aula 4

Modelos Decoder e Encoder-Decoder

De GPT3 gerando texto token a token, à tradução automática combinando um encoder e um decoder com cross-attention.



De gerar palavras a traduzir frases

Na Aula 3, o encoder construiu representações. Hoje o objetivo muda: gerar a próxima palavra — e depois, gerar uma frase inteira em outra língua.

12.7	Decoder: GPT3	modelagem de linguagem	12.8	Encoder-decoder	tradução automática
12.7.2	Masked self-attention	o "truque" do decoder	12.9*	Cross-attention	decoder atende ao encoder
12.7.3	Geração de texto	amostragem autoregressiva			

Representar vs. gerar

Mesma arquitetura de base (embeddings + camadas transformer), objetivo completamente diferente.

ENCODER (AULA 3)

Constrói uma representação do texto inteiro, para depois ser especializada (fine-tuning) em várias tarefas diferentes.

DECODER (HOJE)

Tem um único propósito: prever o próximo token da sequência. Repetindo esse processo, gera texto novo.

GPT3: prever a próxima palavra

Arquitetura muito parecida com a de um encoder — uma série de camadas transformer sobre embeddings aprendidos. O que muda é o objetivo e (como veremos) a forma como a atenção é restringida.

GPT3 é um modelo de linguagem autoregressivo

Dado o começo de uma frase, qual é a distribuição de probabilidade sobre a próxima palavra?

A probabilidade da frase, fatorada

A probabilidade da sequência inteira se decompõe num produto de probabilidades condicionais — cada palavra depende só das anteriores.

exemplo – "It takes great courage to let yourself appear weak"

$$Pr(\text{It}) \times Pr(\text{takes} \mid \text{It}) \times Pr(\text{great} \mid \text{It takes}) \times \dots \times Pr(\text{weak} \mid \text{It} \dots \text{appear})$$

Forma geral:

$$Pr(t_1, \dots, t_N) = Pr(t_1) \prod_{n=2}^N Pr(t_n \mid t_1, \dots, t_{n-1})$$

O problema: "colar" durante o treino

Queremos processar a frase inteira de uma vez (eficiente), calculando todas as probabilidades condicionais no mesmo forward pass — em vez de um passo por token.

- Mas se toda a frase entra de uma vez, o termo $\log Pr(\text{great} \mid \text{It takes})$ teria acesso não só à resposta **great**, como também ao contexto à direita: *courage to let yourself appear weak*.
- O modelo "colaria" em vez de aprender a prever — e nunca treinaria de verdade.

Solução: já que os tokens só interagem entre si nas camadas de self-attention, basta impedir que uma posição atenda às posições futuras.

Como a máscara é aplicada

Antes do softmax, os produtos escalares que ligariam uma posição a tokens futuros são forçados a

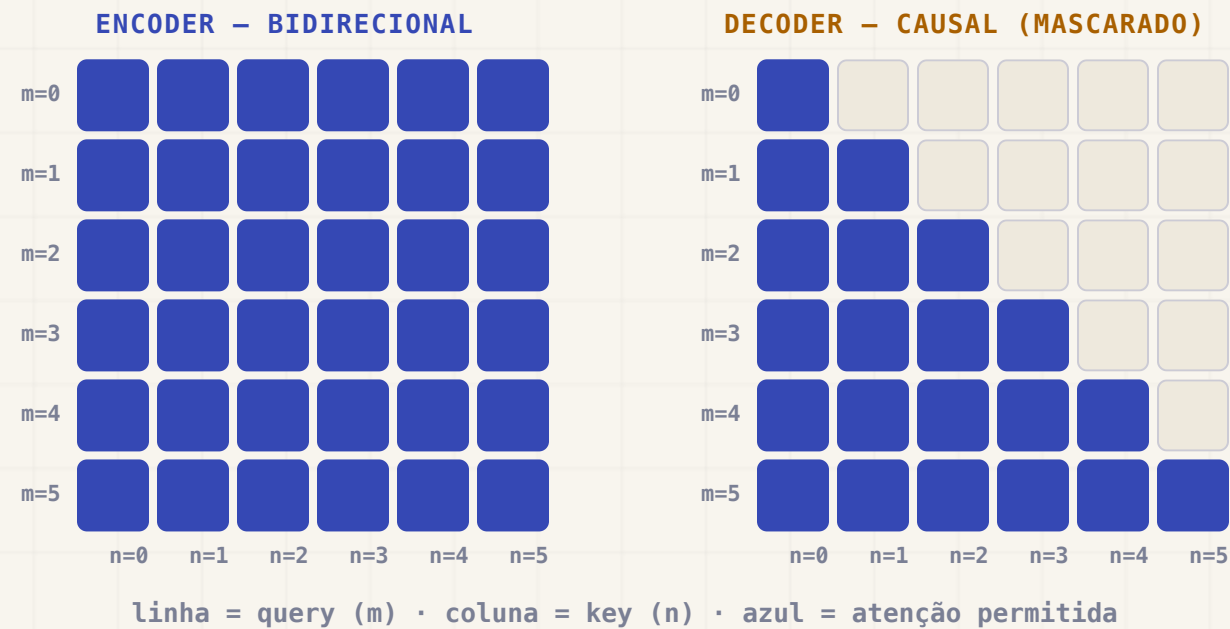
— $-\infty$ — o que faz o softmax atribuir peso zero a eles.

$$a[\mathbf{x}_m, \mathbf{x}_n] = \text{softmax}_m \left[\mathbf{k}_{\bullet}^T \mathbf{q}_n \right], \quad \mathbf{k}_m^T \mathbf{q}_n \leftarrow -\infty \text{ se } m > n$$

m = posição da key (candidata a ser atendida), n = posição da query (quem está consultando). Zerar o peso das posições $m > n$ é exatamente o mesmo mecanismo do softmax — só muda o input antes dele.

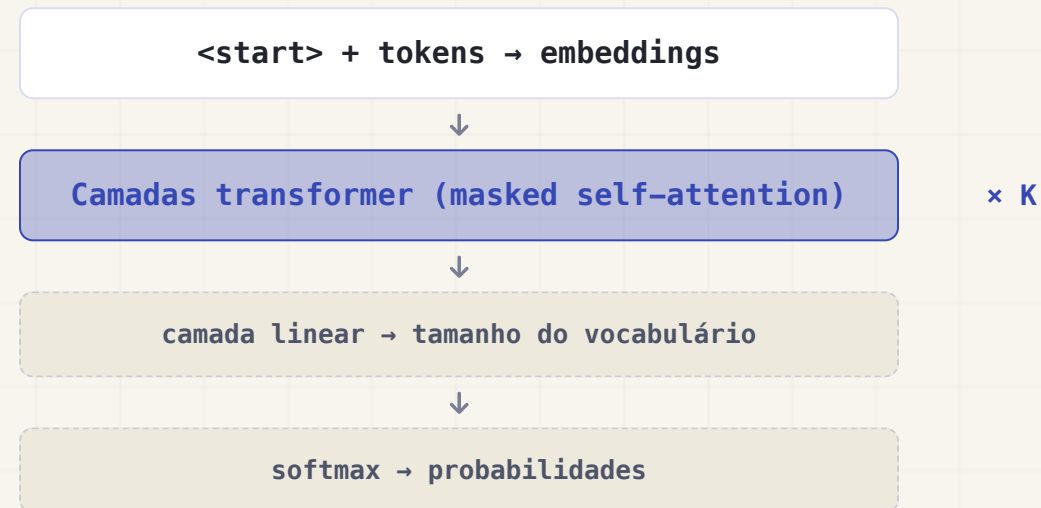
Encoder vs. decoder, lado a lado

A máscara transforma a matriz de interação num triângulo: cada posição só "vê" a si mesma e o que veio antes.



Treinando o decoder, de ponta a ponta

Cada embedding de saída representa "uma frase parcial" — e o objetivo, em cada posição, é prever o próximo token de verdade.



Perda: entropia cruzada multiclasse somada em todas as posições — cada palavra da frase contribui um termo, tornando o treino bem mais eficiente em dados que o mascaramento do encoder.

Gerar: o primeiro modelo generativo do curso

Como o decoder define uma distribuição de probabilidade sobre sequências, dá para amostrar dele — gerando um token, anexando-o à sequência, e repetindo.



ESCOLHENDO O TOKEN

Token mais provável (greedy), amostragem da distribuição, beam search (mantém várias hipóteses) ou top-k sampling (só entre os k mais prováveis).

EFICIÊNCIA

Como o masked self-attention faz os embeddings anteriores não dependerem dos posteriores, boa parte do cálculo já feito pode ser reaproveitado a cada novo token.

GPT3 em escala

SEQUÊNCIA & BATCH

Sequências de 2048 tokens · batch de 3.2 milhões de tokens.

CAMADAS & DIMENSÃO

96 camadas transformer, embeddings de dimensão 12.288.

HEADS

96 heads de self-attention (algumas camadas usam atenção esparsa).

E quando o objetivo é traduzir?

Um decoder puro só continua uma sequência na mesma língua/domínio. Tarefas sequência→sequência — como tradução — precisam condicionar a geração em outra sequência inteira: a frase de origem.

"*The cat sat on the mat.*" → "*Le chat s'est assis sur le tapis.*"

tradução = tarefa sequência-para-sequência

Um encoder para ler, um decoder para escrever

O encoder processa a frase de origem (inglês) e produz uma representação. O decoder gera a frase de destino (francês) token a token, usando masked self-attention — mas também atendendo à saída do encoder.

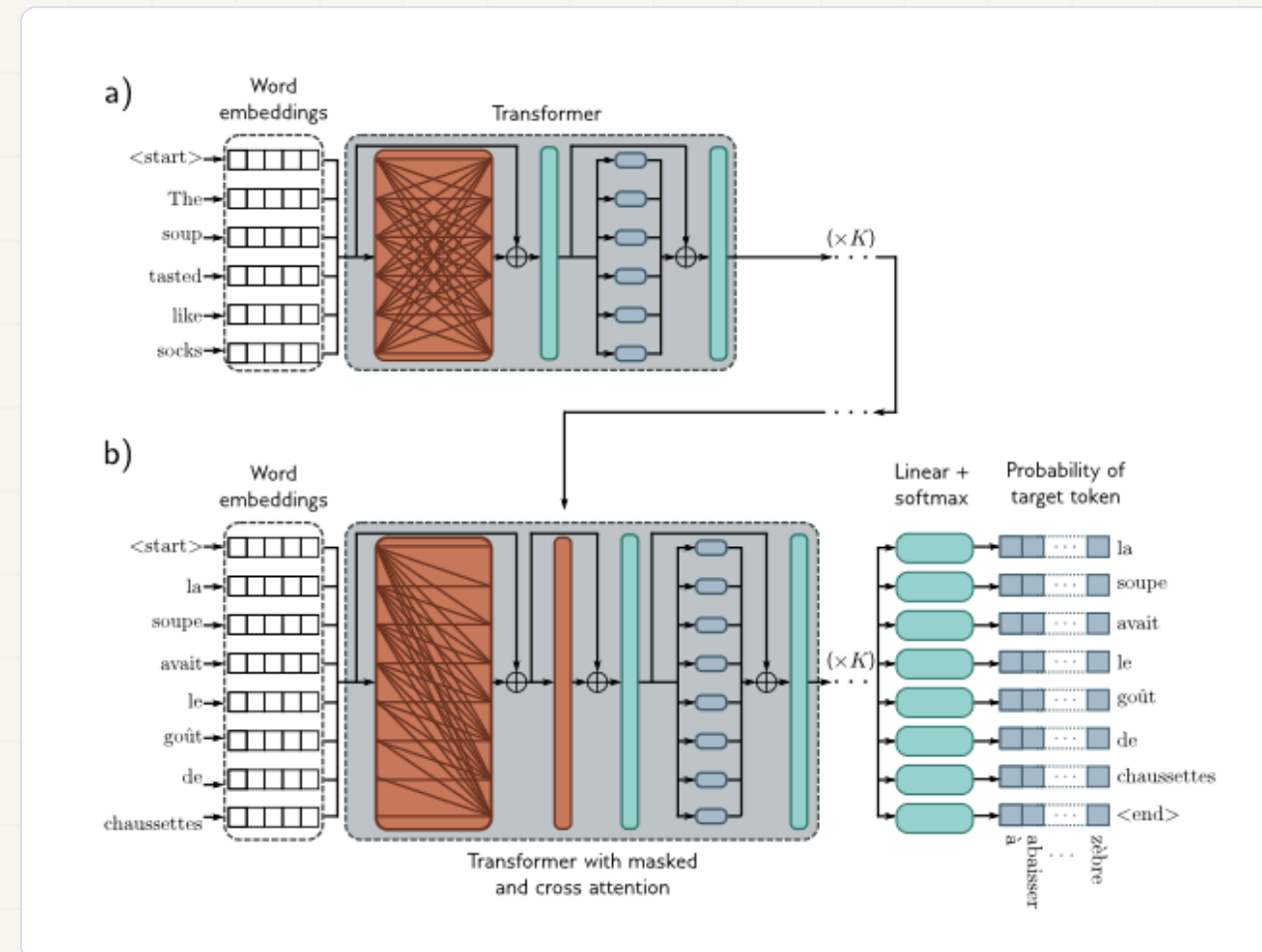


Figura 12.13 – Prince, *Understanding Deep Learning* (2023)

retângulo laranja em b) = a cross-attention, a única conexão nova entre as duas pilhas

Queries do decoder, keys e values do encoder

O fluxo de cálculo é idêntico à self-attention comum — só muda de onde vêm Q, K e V.

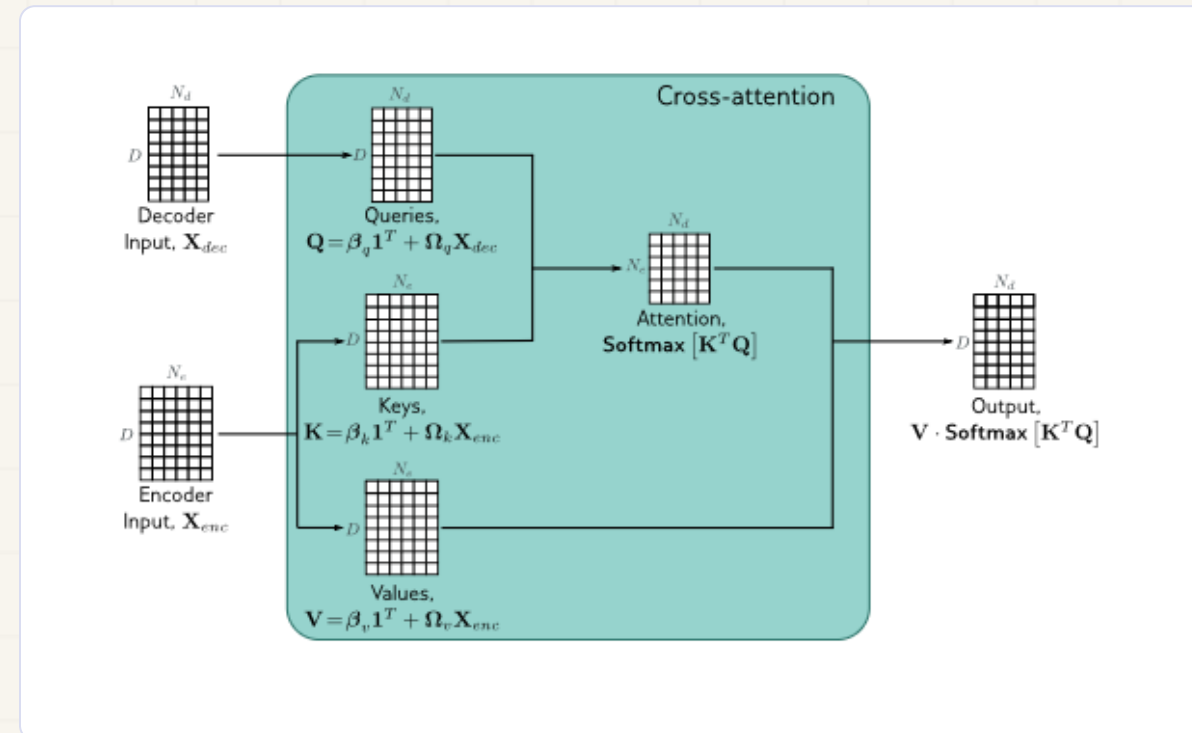


Figura 12.14 – Prince, *Understanding Deep Learning* (2023)

$\mathbf{X}_{dec}$ carrega a estatística da língua de destino; $\mathbf{X}_{enc}$ carrega a estatística da língua de origem. Cada palavra francesa "pergunta" (query) e o inglês "responde" (key/value) — é assim que a tradução fica condicionada na frase de origem inteira.

A camada do decoder, com uma peça a mais

A camada original do decoder (masked self-attention + mlp por token) ganha uma terceira sub-camada no meio.

